

Convocatoria Extraordinaria Julio 2023

- Se debe escribir el nombre, apellidos, número de matrícula y DNI en las hojas de respuesta.
- Para indicar su nombre, utilice letra clara, mejor en mayúsculas.
- Escriba las respuestas en los recuadros de cada apartado. Solo se entregan las hojas con las respuestas.
- Por ordenador, consulta las instrucciones en el apartado "**Se pide**".
- El examen tendrá una duración de **1 hora y media**.

1. Descripción general del problema

El objetivo del programa a desarrollar es la implementación de una calculadora en notación polaca inversa (o notación posfija, en inglés, Reverse Polish Notation, o **RPN**).

En una calculadora *RPN*, la forma de introducir los datos consiste en escribir primero los operandos y luego la operación. Por ejemplo: la suma, $3 + 2$, se escribe como: $3\ 2\ +$. Y de forma análoga el resto de operaciones aritméticas. En todas las operaciones se escribe el primer operando en primer lugar y, respectivamente, en segundo lugar el segundo operando. Por ejemplo: la resta, $3 - 2$, se escribe como: $3\ 2\ -$. Cuando se introducen varios números se guardan en un *array*. Así, por ejemplo: $5 * (3 + 1)$ se escribe como $3\ 1\ +\ 5\ *$, como se puede observar no es necesario el paréntesis. En *RPN* los resultados de las operaciones se guardan, por eso en el ejemplo anterior se puede multiplicar el resultado de la suma por 5. También se guardan los números que todavía no se hayan usado, por ejemplo: la expresión en *RPN*, $2\ 5\ 1\ -\ /\$, es equivalente a $2 / (5 - 1)$, y, de nuevo, no es necesario el paréntesis. Paso a paso sería:

1. Se lee el número 2 y se guarda, el *array* donde se guardan los números queda $[2]$
2. Se lee el número 5 y se guarda, queda $[2\ 5]$
3. Se lee el número 1 y se guarda, queda $[2\ 5\ 1]$
4. Se lee el símbolo de la resta, se realiza la operación $5 - 1$, se quitan ambos números y el resultado se guarda, queda $[2\ 4]$
5. Se lee el símbolo de la división, se realiza la operación $2 / 4$, se quitan ambos números y el resultado se guarda, queda $[0.5]$. Se supone que la calculadora trabaja con **números en coma flotante**.

2. Desarrollo de la implementación

Para implementar la calculadora se ha descompuesto el problema en varias funciones de C que van realizando cada una de las manipulaciones de datos necesarias para resolver el problema en conjunto. Respecto de la definición teórica del problema se van a añadir los siguientes detalles de implementación:

- El usuario de la calculadora podrá elegir entre introducir datos por el canal estándar de entrada o un archivo, en este último caso, indicando su nombre como argumento del programa. El programa acaba cuando se llega al final de archivo, a efectos de la implementación el programa termina si no es capaz de leer un número o una operación o no es capaz de reservar la memoria necesaria para guardar los números.
- Se asume como **precondición** que, al introducir los datos, **siempre hay datos suficientes para poder realizar las operaciones** que se introduzcan con posterioridad. **No hace falta comprobarlo**. Por ejemplo, para la operación suma se puede suponer que ya hay, al menos, 2 números introducidos antes de que se introduzca la operación.
- Para guardar los números se va a utilizar un **array dinámico**, los números se guardan según se van leyendo. El tamaño inicial del *array* será 8, no debe existir un límite al número de elementos en el *array*, si fuese necesario se debe hacer crecer el tamaño del *array* al doble de su tamaño actual para seguir guardando nuevos datos. Para facilitar la gestión del *array* dinámico, se van a agrupar en un **struct**: el puntero al primer elemento del *array* dinámico, el tamaño del *array* y cuántos números hay guardados.

- Para distinguir si el dato introducido es un número o una operación se va a utilizar la capacidad de **fscanf** para indicar si se ha podido leer o no un dato del tipo indicado (ver documentación de **fscanf**). Como es **obligatorio** utilizar **fscanf** para leer tanto operaciones como operandos, la colocación de eventuales saltos de línea en la entrada es irrelevante. El archivo de ejemplo tiene algunos saltos de línea simplemente para facilitar la lectura del mismo no por ningún tipo de requisito por parte del programa.
- Además de las operaciones aritméticas binarias comunes: suma, resta, multiplicación y división, se van a implementar una operación especial: *suma_total*. La operación *suma_total* calcula la suma de todos los elementos en el *array*, vacía el *array* y guarda el resultado. Se puede suponer que, al menos, hay un elemento en el *array*. No es necesario comprobarlo.

3. Pruebas

En el siguiente archivo, *entradas.txt*, se han incluido una serie de casos de prueba básicos para comprobar si las operaciones se realizan correctamente:

```
1 3 5.75 +
2 *
5 /
-
1 6.5 8 -10 8.8 2.3 4.5 5.6
suma_total
```

Al usar este archivo se tienen que producir el siguiente resultado en el canal de salida estándar:

```
Se ha abierto el archivo: 'entradas.txt'
Haciendo: + => Resultado 8.75
Haciendo: * => Resultado 17.50
Haciendo: / => Resultado 3.50
Haciendo: - => Resultado -2.50
Haciendo: suma_total => Resultado 24.20
Se cierra el archivo: 'entradas.txt'
```

Al usar este archivo se tienen que producir el siguiente resultado en el canal de errores estándar:

```
[ 1.00 3.00 5.75 ]
[ 1.00 8.75 2.00 ]
[ 1.00 17.50 5.00 ]
[ 1.00 3.50 ]
[ -2.50 1.00 6.50 8.00 -10.00 8.80 2.30 4.50 5.60 ]
```

4. Se pide

Escribir o completar el **código fuente que falta** en los recuadros.

Si se realiza el examen en ordenador se debe utilizar el archivo auxiliar que se puede encontrar en Moodle. El alumno debe escribir las respuesta entre las marcas

```
/*IniciarRespuesta*/ /*FinalDeRespuesta*/
```

en ningún caso se pueden modificar, eliminar estos comentarios, no se debe editar ningún otro fragmento de código fuera de estas zonas. Se debe entregar el archivo **rpn.c** en triqui con el comando:

```
entrega.pps rpn.c
```

Se recuerda que las opciones de compilación deben ser: **-ansi -pedantic -Wall -Wextra**

5. Funciones de la librería estándar

fopen:

```
FILE *fopen(const char *restrict pathname, const char *restrict mode);
```

The `fopen()` function opens the file whose name is the string pointed to by `pathname` and associates a stream with it.

The argument `mode` points to a string beginning with one of the following sequences (possibly followed by additional characters, as described below):

`r` Open text file for reading. The stream is positioned at the beginning of the file.

RETURN VALUE Upon successful completion `fopen()` return a FILE pointer.

Otherwise, NULL is returned.

fclose:

```
int fclose(FILE *stream);
```

The `fclose()` function [...] closes the underlying file descriptor.

RETURN VALUE Upon successful completion, 0 is returned.

fscanf:

```
int fscanf(FILE *restrict stream, const char *restrict format, ...);
```

The `fscanf()` function shall read from the named input stream. [...] reads bytes, interprets them according to a format, and stores the results in its arguments. Each expects, as arguments, a control string format described below, and a set of pointer arguments indicating where the converted input should be stored. [...]

RETURN VALUE Upon successful completion, these functions shall return the number of successfully matched and assigned input items; this number can be zero in the event of an early matching failure. If the input ends before the first conversion (if any) has completed, and without a matching failure having occurred, EOF shall be returned. If an error occurs before the first conversion (if any) has completed, and without a matching failure having occurred, EOF shall be returned [...].

malloc:

```
void *malloc(size_t size);
```

The `malloc()` function allocates `size` bytes and returns a pointer to the allocated memory.

The memory is not initialized. [...]

RETURN VALUE The `malloc()`, `calloc()`, `realloc()`, and `reallocarray()` functions return a pointer to the allocated memory [...]. On error, these functions return NULL [...]

realloc:

```
void *realloc(void *ptr, size_t size);
```

The `realloc()` function changes the size of the memory block pointed to by `ptr` to `size` bytes. The contents of the memory will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will not be initialized. [...]

Unless `ptr` is NULL, it must have been returned by an earlier call to `malloc` or related functions. If the area pointed to was moved, a `free(ptr)` is done.

RETURN VALUE The `malloc()`, `calloc()`, `realloc()`, and `reallocarray()` functions return a pointer to the allocated memory [...]. On error, these functions return NULL [...]

strcmp:

```
int strcmp(const char *s1, const char *s2);
```

The `strcmp()` function compares the two strings `s1` and `s2`. [...] The comparison is done using unsigned characters.

`strcmp()` returns an integer indicating the result of the comparison, as follows:

- 0, if the `s1` and `s2` are equal;
- a negative value if `s1` is less than `s2`;
- a positive value if `s1` is greater than `s2`.

HOJAS DE RESPUESTAS

DNI:

Apellidos, Nombre:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void procesar_operaciones(FILE *); /* Lee el archivo y lo procesa como una calculadora en RPN */
```

/ A) Abre el archivo dado como argumento y llama a la funcion procesar_operaciones.*

B) Escribe un mensaje en el canal de salida estandar indicando si el archivo se puede abrir o no y que se cierra. Los textos a mostrar son:

"Se ha abierto el archivo: '%s'\n"

"Se cierra el archivo: '%s'\n"

"No se puede abrir el archivo: '%s'\n"

*C) Si el archivo no se puede abrir ok debe tomar el valor 1. */*

```
int main(int argc, char *argv[]) {
```

```
    int ok = 0;
```

```
    if (argc > 1) {
```

Apartado 1. (1 punto)

```
        char *filename = argv[1];
        FILE *f = fopen(filename, "r");
        if (f) {
            printf("Se ha abierto el archivo: '%s'\n", filename);
            procesar_operaciones(f);
            printf("Se cierra el archivo: '%s'\n", filename);
            fclose(f);
        }
        else {
            printf("No se puede abrir el archivo: '%s'\n", filename);
            ok = 1;
        }
    }
```

```
}
```

```
else
```

```
    procesar_operaciones(stdin);
```

```
return ok;
```

```
}
```

/ A) Intenta leer un numero en coma flotante del archivo y asignar su valor al double apuntado por num, si es posible devuelve 1.*

B) En caso contrario, intenta leer una cadena de hasta 31 caracteres y guardar su valor en el string apuntado por operacion, si es posible devuelve 2.

C) En otro caso devuelve 0.

Nota: El formato para leer un double es "%lf", "%31s" para un string hasta 31 chars de longitud

**/*

```
int leer_numero_u_operacion(FILE* f, double *num, char *operacion) {
```

Apartado 2. (1 punto)

```
    int leído = 0;
    if (fscanf(f, "%lf", num) == 1) {
        leído = 1;
    }
    else if (fscanf(f, "%31s", operacion) == 1) {
        leído = 2;
    }
    return leído;
```

```
}
```

```
typedef struct {
```

```
    double *array; /* Direccion del primer elemento del array dinamico */
```

```

    unsigned tam; /* numero maximo de elementos en el array */
    unsigned len; /* indica cuantos numeros se han guardado en el array dinamico */
} rpn_t; /* Esta estructura sirve para guardar los numeros que se introducen y los resultados */

```

```

/* A) Fija tam a 8 y la longitud actual a cero.
   B) Reserva memoria dinamica para 8 numeros en coma flotante
   C) Si no se puede reservar memoria se debe escribir "No se puede reservar memoria."
   D) y retornar 0. En caso contrario retornar 1.
*/

```

```

int inicializar(rpn_t* ops) {

```

Apartado 3. (1 puntos)

```

    ops->tam = 8;
    ops->len = 0;
    ops->array = malloc(ops->tam*sizeof(double));
    if ( ops->array == NULL ) {
        printf("No se puede reservar memoria.");
        return 0;
    }
    return 1;

```

```

}
/* A) Comprueba si el numero a guardar cabe en el array, si NO es asi:
   B) Usa realloc para redimensionar el array tomando como nuevo tamaño el doble del anterior
   C) Coloca num en el array como ultimo elemento.
   D) Si no se puede reservar memoria se debe escribir: "No se puede reservar mas memoria."
   E) y retornar 0. En caso contrario retornar 1.
*/

```

```

int guardar_numero(double num, rpn_t *ops) {

```

Apartado 4. (2 puntos)

```

    if (ops->len == ops->tam)
    {
        void *aux;
        ops->tam *= 2;
        aux = realloc(ops->array, ops->tam*sizeof(double));
        if (aux == NULL)
        {
            printf("No se puede reservar mas memoria.");
            return 0;
        }
        ops->array = aux;
    }
    ops->array[ops->len] = num;
    ops->len += 1;
    return 1;

```

```

}
/* Suma todos los numeros *guardados* en el array y devuelve el resultado. */
double suma_todos(rpn_t *ops) {

```

Apartado 5. (1 punto)

```

    double suma = 0;
    unsigned i;
    for ( i = 0; i < ops->len; ++i ) {
        suma += ops->array[i];
    }
    return suma;

```

```

}

/* Se usa esta funcion para mostrar los numeros introducidos */
void mostrar(rpn_t* ops) {
    unsigned i;
    fprintf(stderr, "[_");

```

```

    for ( i = 0; i < ops->len; ++i ) { fprintf(stderr, "%.2f_", ops->array[i]); }
    fprintf(stderr, "]\n");
}

```

/ Funcion auxiliar para obtener el ultimo elemento en el array (indice: len-1) */*

```

double ultimo(rpn_t* ops) {
    return ops->array[ops->len - 1];
}

```

/ Comprueba la operacion a realizar y la aplica a los numeros guardados.*

A) Comprueba cual es la operacion, los valores posibles son: "+", "-", "", "/" y "suma_total".*

B) La operacion "suma_total" suma todos los valores guardados. Es obligatorio usar la funcion suma_todos.

C) Precondicion: Se supone que, al menos, hay dos numeros guardados para las operaciones binarias y uno para suma_total.

Nota: El codigo ya suministrado ya escribe tanto la operacion y su resultado como los valores en el array dinamico.

**/*

```

void realizar_operacion(char *operacion, rpn_t *ops) {
    mostrar(ops);
    printf("Haciendo: %s=>", operacion);

```

Apartado 6. (2 puntos)

```

    if (strcmp(operacion, "+") == 0)
    {
        ops->array[ops->len - 2] += ops->array[ops->len - 1];
        ops->len -= 1;
    }
    else if (strcmp(operacion, "-") == 0)
    {
        ops->array[ops->len - 2] -= ops->array[ops->len - 1];
        ops->len -= 1;
    }
    else if (strcmp(operacion, "*") == 0)
    {
        ops->array[ops->len - 2] *= ops->array[ops->len - 1];
        ops->len -= 1;
    }
    else if (strcmp(operacion, "/") == 0)
    {
        ops->array[ops->len - 2] /= ops->array[ops->len - 1];
        ops->len -= 1;
    }
    else if (strcmp(operacion, "suma_total") == 0)
    {
        ops->array[0] = suma_todos(ops);
        ops->len = 1;
    }
}

```

```

printf("Resultado %.2f\n", ultimo(ops));

```

```

}

```

/ Es OBLIGATORIO llamar a esta funcion para liberar el array dinamico */*

```
void liberar(rpn_t *ops)
{
    if (ops->array)
        free(ops->array);
}
```

/ Lee datos desde el stream f, guarda los numeros y realiza las operaciones.*

A) Declara e inicializa el struct con el array donde se van a guardar los numeros. Se pueden declarar otras variables (de hecho, alguna mas se necesita).

B) Usa, OBLIGATORIAMENTE, la funcion leer_numero_u_operacion para leer el numero u operacion desde el archivo.

C) Si se lee un numero se guarda en el array. Es obligatorio usar la funcion guardar_numero.

D) Si se lee una operacion se usa la funcion realizar_operacion para aplicarla. Es obligatorio usar la funcion realizar_operacion.

E) En cualquier otro caso la funcion debe terminar.

F) La funcion tambien debe terminar si no se puede inicializar el array dinamico o no se pueda guardar un numero porque no haya memoria suficiente.

G) Liberar la memoria dinamica usando la funcion liberar.

**/*

```
void procesar_operaciones(FILE *f) {
    double num = 0;
    char operacion[32];
```

Apartado 7. (2 puntos)

```
int leido, ok;
rpn_t ops;
ok = inicializar(&ops);
leido = leer_numero_u_operacion(f, &num, operacion);
while ( leido && ok ) {
    if ( leido == 1 ) {
        ok = guardar_numero(num, &ops);
    } else {
        realizar_operacion(operacion, &ops);
    }
    leido = leer_numero_u_operacion(f, &num, operacion);
}
liberar(&ops);
```

```
}
```